

Ring of the Middle Earth — Architecture Document

Course: Distributed Application Development **Option:** B — Go + Kafka

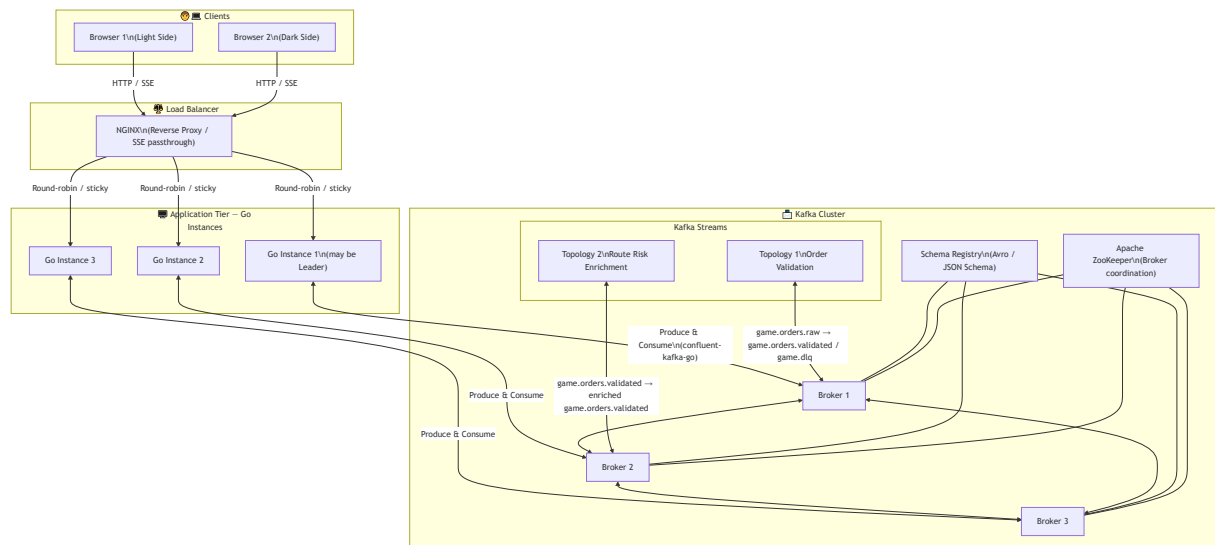
Date: May 25, 2026 **Prepared By:** Melek Başpınar, Tuğrul Akgül

Table of Contents

1. System Diagram
2. Goroutine Map
3. Kafka Topic Design
4. Paradigm Justification
5. Reflection
6. LLM Usage Log

1. System Diagram

The following diagram shows the complete high-level topology of the Ring of the Middle Earth system, from browser clients through the load balancer, the Go application tier, the Kafka cluster, and the supporting infrastructure.



Component Responsibilities

Component	Role
NGINX	Reverse proxy; passes SSE connections through without buffering (<code>proxy_buffering off</code>); routes REST calls to Go instances
Go Instance (Leader)	Runs <code>TurnProcessor</code> ; owns turn advancement; operates work Kafka consumer for <code>game.orders.validated</code>
Go Instance (Follower)	Handles HTTP requests; projects state from Kafka; serves SSE streams; participates in leader election
Kafka Broker ×3	Durable event log; partition-level replication; consumer group rebalancing
Kafka Streams Topology 1	Validates raw orders against KTable snapshots; emits to validated or DLQ
Kafka Streams Topology 2	Enriches validated <code>ASSIGN_ROUTE/REDIRECT</code> orders with route risk scores
Schema Registry	Central schema store; enforces backward-compatible schema evolution
ZooKeeper	Stores Kafka broker metadata, controller election, ACLs

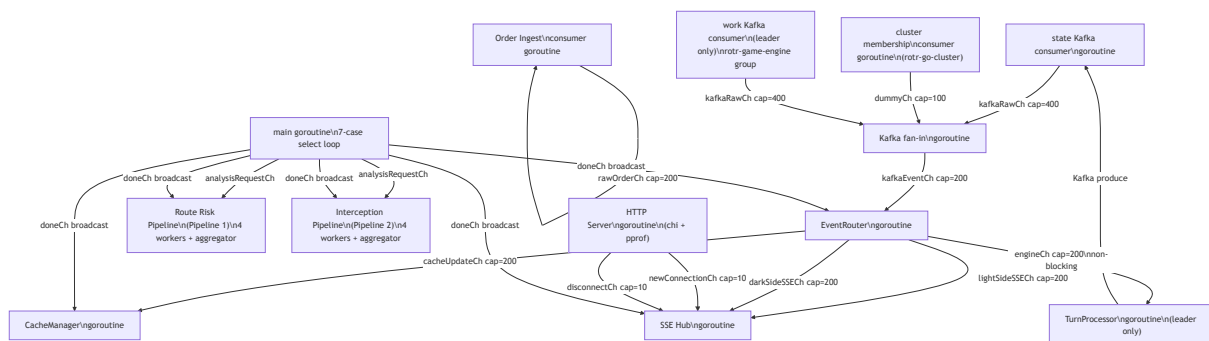
2. Goroutine Map

This section documents **every goroutine** in a running Go instance, its input/output channels, buffer capacities, and lifecycle ownership.

2.1 Channel Inventory

CHANNEL DIRECTORY		
Channel Name	Capacity	Direction / Purpose
kafkaRawCh	400	Kafka consumer → fan-in goroutine
dummyCh	100	cluster membership drain sink
kafkaEventCh	200	fan-in → EventRouter
lightSideSSECh	200	EventRouter → SSE Hub (light)
darkSideSSECh	200	EventRouter → SSE Hub (dark)
cacheUpdateCh	200	EventRouter → CacheManager
engineCh	200	EventRouter → TurnProcessor (leader)
analysisRequestCh	—	main select → Pipeline dispatchers
workCh (Pipeline 1)	20	dispatcher → 4 route-risk workers
workCh (Pipeline 2)	30	dispatcher → 4 interception workers
newConnectionCh	10	HTTP handler → SSE Hub (register)
disconnectCh	10	HTTP handler → SSE Hub (unregister)
rawOrderKafkaCh	200	order-ingest Kafka consumer output
rawOrderCh	200	ingest → RunOrderIngest validator
signalCh	1	OS signal → main (os.Signal)
doneCh	—	close-broadcast shutdown sentinel

2.2 Goroutine Dependency Graph



2.3 main Goroutine — 7-Case Select Loop

The main goroutine is the application's central control loop. It runs a `select` over seven channels simultaneously and never blocks on any single path.

```

for {
    select {
        case req := <-analysisRequestCh:    // Case 1
        case <-turnTicker.C:                // Case 2
        case <-electionTicker.C:           // Case 3
        case sig := <-signalCh:             // Case 4
        case <-doneCh:                      // Case 5
        case <-stateLogTicker.C:            // Case 6
        case <-goroutineMonitorTicker.C:    // Case 7
    }
}

```

Case	Source	Action
1	analysisRequestCh	Dispatch incoming analysis request to the appropriate pipeline (Route Risk or Interception)
2	turnTicker.C (60 s)	Trigger turn advancement — leader only ; no-op on followers
3	electionTicker.C (3 s)	Send or refresh leader heartbeat; promote self if no heartbeat seen within timeout
4	signalCh	Receive SIGINT/SIGTERM; initiate graceful shutdown (close doneCh, drain channels)
5	doneCh	Observe programmatic shutdown request; exit the select loop cleanly
6	stateLogTicker.C (30 s)	Log current WorldStateCache snapshot for observability / debug
7	goroutineMonitorTicker.C (60 s)	Call runtime.NumGoroutine(); emit WARN if count > 100 (goroutine leak detection)

2.4 State Kafka Consumer Goroutine

Input: Kafka broker (topic group: all state topics, all partitions)

Output: kafkaRawCh (cap = 400)

Lifecycle: starts before main loop; observes doneCh for clean exit

- Subscribes to game.events.*, game.broadcast, game.ring.*, game.session
- Decodes message envelope, wraps into internal KafkaEvent struct
- Sends to kafkaRawCh; if channel is full, records a dropped-message counter (non-blocking send with default case)

2.5 Cluster Membership Consumer Goroutine

Input: Kafka broker (topic: rotr-go-cluster, consumer group: unique per instance)

Output: dummyCh (cap = 100) – drained and discarded

Lifecycle: always running; used only for membership presence signalling



- Each Go instance publishes a heartbeat to `rotr-go-cluster`
- Consuming one's own messages keeps the consumer group active
- `dummyCh` prevents the consumer goroutine from blocking; messages are dropped intentionally

2.6 Kafka Fan-In Goroutine

Input: `kafkaRawCh` (cap = 400) [from state consumer + leader work consumer]

Output: `kafkaEventCh` (cap = 200)

Lifecycle: observes `doneCh`

- Merges the two upstream Kafka consumer streams into one `kafkaEventCh`
- Applies lightweight de-duplication (offset-based) to prevent double-processing on rebalance
- Preserves original Kafka offset and partition metadata for downstream use

2.7 EventRouter Goroutine

Input: `kafkaEventCh` (cap = 200)

Outputs:

`lightSideSSECh` (cap = 200) – light-side SSE push

`darkSideSSECh` (cap = 200) – dark-side SSE push [ring bearer stripped]

`cacheUpdateCh` (cap = 200) – cache write

`engineCh` (cap = 200) – turn processor (non-blocking, drops if full)

Lifecycle: observes `doneCh`

Routing Table:

Topic / Event Type	<code>lightSideSSECh</code>	<code>darkSideSSECh</code>	<code>cacheUpdateCh</code>	<code>engineCh</code>
<code>game.ring.position</code>	✓	✗	✓	✗
<code>game.ring.detection</code>	✗	✓	✓	✗
<code>game.broadcast</code>	✓	✓ (stripped)	✓	✗
<code>game.events.*</code>	✓	✓	✓	✗
<code>game.orders.validated</code>	✗	✗	✗	✓ (non-blocking)
<code>game.session</code>	✗	✗	✓	✗

Information boundary: `game.ring.position` (the ring bearer's true location) is **never** routed to `darkSideSSECh`. The `game.broadcast` events routed to dark-side clients pass through `stripRingBearer()`, which removes the `trueRegion` field before dispatch.

```
// Non-blocking send to engineCh – drops if TurnProcessor is busy
select {
case engineCh <- event:
default:
    metrics.DroppedEngineEvents.Inc()
}
```

2.8 CacheManager Goroutine

Input: cacheUpdateCh (cap = 200)
Owns: WorldStateCache (in-memory; read via RWMutex-protected helper functions)
Lifecycle: observes doneCh; flushes cache snapshot on shutdown

- Processes cache update events sequentially (single goroutine owns the write lock)
- Exposes read-only helper functions used by HTTP handlers and SSE Hub
- Maintains two views: fullState (for leader/light-side) and strippedState (for dark-side API responses)

2.9 SSE Hub Goroutine

Inputs:

lightSideSSECh (cap = 200)
darkSideSSECh (cap = 200)
newConnectionCh (cap = 10) – register new SSE client
disconnectCh (cap = 10) – unregister client

Lifecycle: observes doneCh; sends SSE `close` frame to all clients on shutdown

- Maintains two client registries: lightClients and darkClients (maps of http.ResponseWriter)
- On event received: fans out to all registered clients of the corresponding side
- On client disconnect/timeout: removes from registry without blocking event flow
- Uses http.Flusher interface to push SSE frames immediately to NGINX

2.10 Route Risk Pipeline — Pipeline 1

Trigger: analysisRequestCh → main goroutine case 1

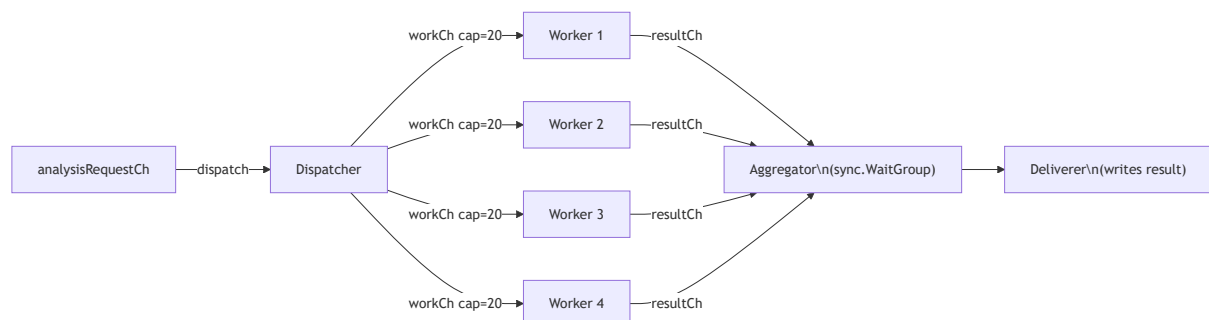
workCh: cap = 20

Workers: 4 goroutines

Timeout: context.Context with 2 s deadline

Sync: sync.WaitGroup across all 4 workers

Stages: dispatcher → workCh → [worker×4] → aggregator → deliverer



- Computes **route risk scores** for all active unit movement orders
- Workers operate on independent path segments; aggregator merges partial scores
- Context cancellation after 2 s ensures the pipeline does not block the main loop

2.11 Interception Pipeline — Pipeline 2

Trigger: analysisRequestCh → main goroutine case 1
 workCh: cap = 30
 Workers: 4 goroutines
 Timeout: context.Context with 2 s deadline
 Sync: sync.WaitGroup across all 4 workers
 Stages: dispatcher → workCh → [worker×4] → aggregator → deliverer

- Computes **interception probabilities** for dark-side units threatening ring bearer path
- Larger workCh capacity (30 vs 20) because interception analysis fans out over more path combinations
- Same WaitGroup + context pattern as Pipeline 1; independent goroutine pool

2.12 Order Ingest Consumer Goroutine

Input (Kafka): rawOrderKafkaCh (cap = 200)
 Internal: rawOrderCh (cap = 200)
 Validation: RunOrderIngest() – 8-rule validator
 Output (Kafka produce):
 game.orders.validated – on success
 game.dlq – on validation failure
 Lifecycle: observes doneCh; commits offsets only after successful produce

8 Validation Rules checked by RunOrderIngest:

Rule ID	Check
WRONG_TURN	Order submitted for the wrong turn number
NOT_YOUR_UNIT	Player does not own the referenced unit
PATH_BLOCKED	Target path is currently blocked
INVALID_PATH	Path ID does not exist in the map graph
UNIT_NOT_ADJACENT	Unit is not adjacent to the path's start node
INVALID_TARGET	Target region/unit is invalid for this order type
ABILITY_ON_COOLDOWN	Special ability not yet available (cooldown not expired)
DUPLICATE_UNIT_ORDER	A prior order for this unit already exists this turn

2.13 HTTP Server Goroutine

Framework: chi router
 Extras: pprof registered at /debug/pprof/*
 Lifecycle: http.Server with graceful shutdown on doneCh

- Exposes REST endpoints for order submission, state queries, session management
- SSE endpoints call into SSE Hub via `newConnectionCh/disconnectCh`
- All read handlers call `CacheManager` helpers under `RLock`
- `/debug/pprof/goroutine` endpoint cross-references `goroutine` monitor metric

2.14 Leader Runtime Goroutines (elected leader only)

When a Go instance wins the leader election (via heartbeat + session topic mechanism), it spawns two additional goroutines:

2.14.1 Work Kafka Consumer

```
Consumer group: rotr-game-engine
Topic:          game.orders.validated
Output:         kafkaRawCh (cap = 400) - injected into fan-in
Lifecycle:      stopped when leadership is lost; doneCh for shutdown
```

- Uses **manual offset commit** (`commitSync` after `TurnProcessor` confirms processing)
- Sticky partition assignment prevents rebalance storms during turn processing

2.14.2 TurnProcessor Goroutine

```
Input:  engineCh (cap = 200)
Output: Kafka produce → game.events.*, game.broadcast, game.ring.*, game.session
Lifecycle: runs only on leader; observes doneCh
```



13-step turn processing sequence:

Step	Action
1	Lock turn mutex; read current turn number from <code>game.session</code>
2	Drain all remaining <code>engineCh</code> messages for the current turn
3	Resolve unit movement orders by path priority
4	Check adjacency and path availability constraints
5	Detect ring bearer interceptions by dark-side units
6	Apply ability effects (buffs, blocks, reveals)
7	Update region control based on movement outcomes
8	Compute ring bearer visibility (true position vs. detected position)
9	Evaluate win conditions (ring reaches Mordor / ring bearer captured)
10	Produce <code>game.events.unit</code> for each unit state change
11	Produce <code>game.events.region</code> for each region control change
12	Produce <code>game.events.path</code> for path status changes
13	Produce <code>game.broadcast</code> + advance turn in <code>game.session</code> ; commit offset

GameOver safety: Step 9 sets a sticky GameOverFlag in `game.session` (compacted topic). Even after leader failover, the new leader reads this flag and skips turn advancement, preventing duplicate GameOver events.

2.15 Goroutine Lifecycle & Termination

Shutdown sequence (triggered by Case 4 or Case 5 in main select):

1. `close(doneCh)` – broadcast to all goroutines
2. HTTP server: `http.Server.Shutdown()` – drain in-flight HTTP requests
3. SSE Hub: send SSE "close" to clients, close `newConnectionCh/disconnectCh`
4. Pipelines: `context.Cancel()`; wait on `sync.WaitGroup`
5. EventRouter: drain `kafkaEventCh`, exit on `doneCh`
6. CacheManager: flush snapshot, exit on `doneCh`
7. Kafka consumers: `Close()` consumer handles, commit offsets
8. main goroutine: exits

Goroutine leak detection (Case 7, every 60 s):

```
n := runtime.NumGoroutine()
if n > 100 {
    log.Warnf("Possible goroutine leak: %d goroutines active", n)
    metrics.GoroutineLeakWarnings.Inc()
}
```

3. Kafka Topic Design

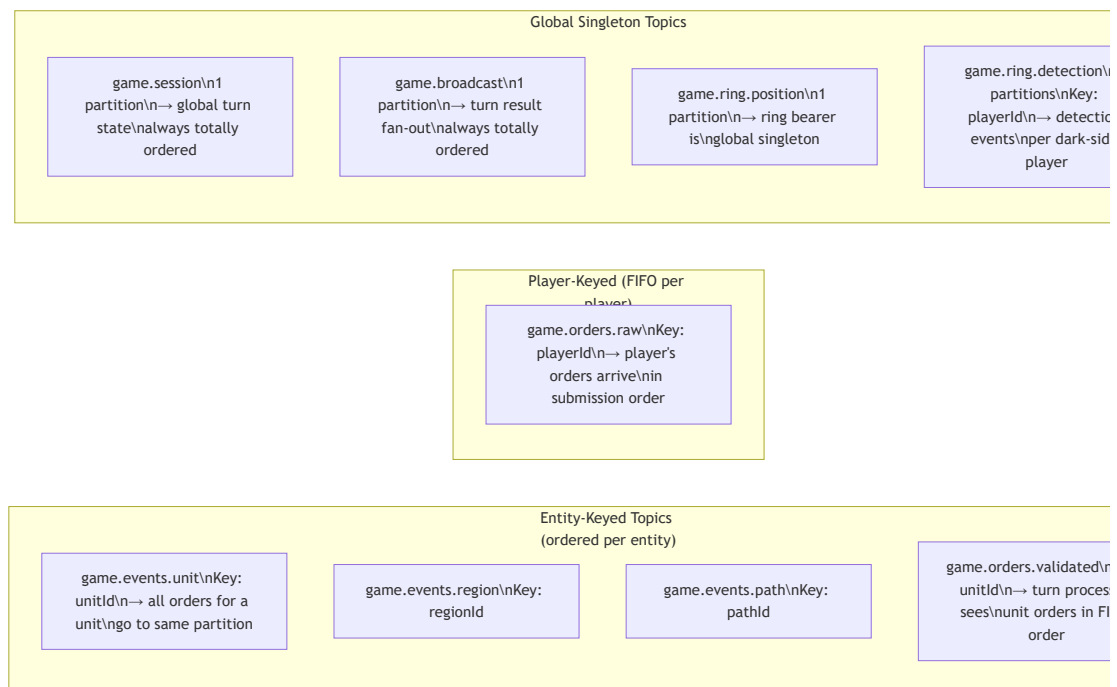
3.1 Topic Table

Topic	Partitions	Partition Key	Cleanup Policy	Retention	Producer	Consumer
<code>game.orders.raw</code>	3	<code>playerId</code>	delete	1 h	Go HTTP API	Kafka Streams Topology 1
<code>game.orders.validated</code>	6	<code>unitId</code>	delete	1 h	Topology 1, Topology 2	Go work consumer (leader)
<code>game.events.unit</code>	6	<code>unitId</code>	delete	7 d	Go TurnProcessor	All Go state consumers, Streams stores
<code>game.events.region</code>	6	<code>regionId</code>	delete	7 d	Go TurnProcessor	All Go state consumers
<code>game.events.path</code>	6	<code>pathId</code>	delete	7 d	Go TurnProcessor	All Go state consumers

Topic	Partitions	Partition Key	Cleanup Policy	Retention	Producer	Consumer
game.session	1	session	compact	∞	Go leader / session heartbeat	All Go instances, Streams validation
game.broadcast	1	(none)	delete	1 h	Go TurnProcessor	All Go state consumers, SSE EventRouter
game.ring.position	1	rbId	delete	1 h	Go TurnProcessor	Light-side only via EventRouter
game.ring.detection	2	playerId	delete	1 h	Go TurnProcessor	Dark-side only via EventRouter
game.dlq	3	errorCode	delete	7 d	Kafka Streams validation	Inspection / monitoring

Total: 10 topics

3.2 Partition Key Rationale

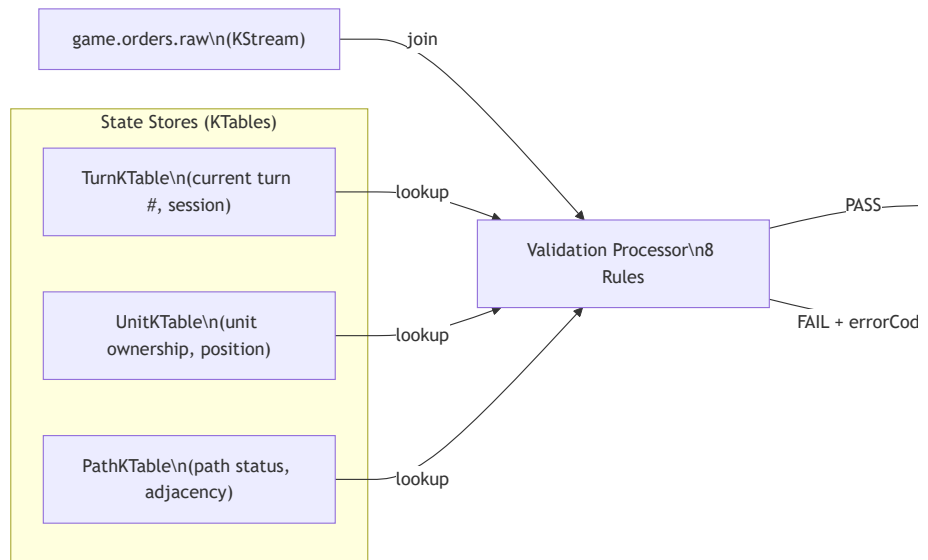


Key Strategy	Reasoning
unitId / regionId / pathId	Same entity always routes to the same partition → consumers process an entity's events in emission order without cross-partition synchronisation
playerId (raw orders)	A single player's orders are partitioned together → FIFO order per player is guaranteed
unitId (validated)	TurnProcessor receives all orders for a unit in submission order → deterministic resolution of conflicting orders
Single partition (session, broadcast, ring position)	These are global state; total ordering across the entire topic is required; multiple partitions would require application-level merge
2 partitions (ring detection)	Detection events are keyed per dark-side player; 2 partitions allows mild parallelism while keeping per-player order

3.3 Retention & Cleanup Policy Rationale

Policy	Topics	Reason
delete / 1 h	orders.raw, orders.validated, broadcast, ring.*	Short-lived operational messages; once processed by the current turn they have no replay value
delete / 7 d	events.unit, events.region, events.path, dlq	Full event history needed for state reconstruction after crash; DLQ kept for manual inspection
compact / ∞	session	Must survive indefinitely; compaction keeps only the latest value per key → new leaders always bootstrap from the latest session record

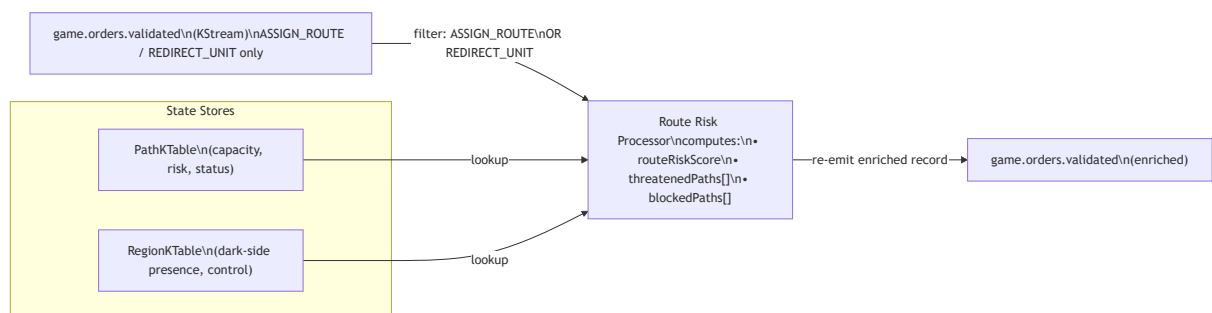
3.4 Kafka Streams Topology 1 — Order Validation



Rules evaluated in order:

1. WRONG_TURN — checked against TurnKTable
2. NOT_YOUR_UNIT — checked against UnitKTable (owner field)
3. PATH_BLOCKED — checked against PathKTable (blocked flag)
4. INVALID_PATH — checked against PathKTable (exists flag)
5. UNIT_NOT_ADJACENT — checked against UnitKTable.position + PathKTable.startNode
6. INVALID_TARGET — semantic check on order type vs. target type
7. ABILITY_ON_COOLDOWN — checked against UnitKTable.abilityCooldown
8. DUPLICATE_UNIT_ORDER — checked against in-flight order deduplication state store

3.5 Kafka Streams Topology 2 — Route Risk Enrichment



- Only `ASSIGN_ROUTE` and `REDIRECT_UNIT` order types pass the filter; all others are passed through unchanged
 - `routeRiskScore` is a `[0.0, 1.0]` float representing the aggregate interception probability along the planned route
 - `threatenedPaths` lists path IDs with dark-side units present
 - `blockedPaths` lists path IDs currently blocked by enemy ability effects
-

4. Paradigm Justification

Q1 — Why Go + Kafka is Well-Suited for This Problem

The "Ring of the Middle Earth" game decomposes naturally into six distinct concerns:

1. **Event ingestion** — players submit orders via HTTP POST
2. **Validation** — orders must be checked against current game state before being accepted
3. **State projection** — all three Go instances must maintain a consistent in-memory view of game state
4. **Turn execution** — exactly one instance must advance the game world each turn
5. **Push delivery** — browser clients must receive real-time updates with information hiding per faction
6. **Parallel analysis** — route risk and interception calculations can proceed concurrently

Kafka addresses concerns 1–4 elegantly: the ordered, durable, partitioned log is an ideal fit for a turn-based game where event replay must reproduce the exact game state. After a crash, a new leader simply replays from the last committed offset on `game.events.*` and the compacted `game.session` topic to restore full state without an external database.

Consumer group rebalancing gives fault tolerance with zero application-layer coordination: when a Go instance dies, Kafka redistributes its partitions to surviving instances within seconds. The application never needs to coordinate partition ownership manually.

KTable state stores in Kafka Streams allow the validation layer to operate as a **stateless application tier** from the Go perspective. Topology 1 owns the validation state; Go instances only need to consume the already-validated stream, which simplifies Go application logic considerably.

Go goroutines make the concurrent architecture readable and cheap. A typical goroutine costs ~2 KB of stack; running 20–30 goroutines per instance is trivially inexpensive. The channel-based communication model maps directly onto the Kafka consumer→fan-in→router→sink pipeline, making the data flow explicit and auditable. Context-based cancellation and `sync.WaitGroup` boundaries give structured lifecycle management without the complexity of an actor framework.

The SSE push model (Case 5 in §2.3) requires long-lived, low-latency connections. Go's `http.Flusher` interface and goroutine-per-connection model handle this naturally. Kafka's consumer group mechanism ensures that even if the SSE-serving instance changes (due to NGINX upstream change or failover), the new instance reads the same event stream from the same offset.

Q2 — What is Genuinely Harder with Go + Kafka

Distributed ownership of turn execution is the most significant complexity. Three Go instances sit behind a round-robin load balancer with no shared memory. Naively, all three could attempt to advance the turn when `turnTicker.C` fires, producing duplicate or conflicting `game.broadcast` events. The solution — a compacted `game.session` topic plus leader heartbeat plus manual offsets — works, but it required careful design:

- The leader must refresh its heartbeat within 3 s ($2 \times$ the `electionTicker` period); if it misses two heartbeats, followers assume the leader is dead and initiate election
- Manual offset commits on `rotr-game-engine` ensure that if a leader crashes mid-turn, the new leader re-processes the same validated orders batch; this requires **exactly-once guarantees** for `GameOver` specifically (the sticky flag in `game.session` prevents duplicate end-of-game events)
- During leader handover, there is a brief window where `engineCh` in the new leader is empty but the old leader's in-flight turn produce has not yet appeared on `game.events.*`; this window must be covered by the new leader waiting for an offset watermark before advancing

Information boundary enforcement must be maintained explicitly at every data path. Unlike an actor model where message protocol boundaries are structural, in Go+Kafka every route in `EventRouter`, every cache helper, every API response handler must individually verify that dark-side clients cannot observe the ring bearer's true position. There is no compile-time guarantee; the boundary is enforced by convention and tested at the integration level. A single forgotten routing rule would silently expose `trueRegion` to dark-side clients.

Goroutine failure handling has no built-in supervision. If `CacheManager` panics, it simply disappears — no automatic restart, no parent notification. Every long-running goroutine must wrap its body in `recover()` and signal `doneCh` on unrecoverable failure, or be wrapped in a retry loop. This is boilerplate that an actor framework handles by default.

Q3 — How Akka Would Solve the Two Hardest Parts

Distributed ownership of turn execution:

In Akka Cluster, the `WorldStateActor` would be deployed as a `ClusterSingleton`. The Cluster Singleton pattern guarantees exactly one instance of the actor is running across the entire cluster at any time. When the node hosting the singleton dies, Akka's Cluster Singleton Manager automatically starts a new instance on another node and replays the actor's durable state via Akka Persistence (event sourcing with a journal). There are no manual heartbeats, no leader election loops, and no offset management — the framework provides all of this transparently. The turn ticker would be an `Akka.Timers` message scheduled inside the singleton actor, and advancement would simply be a message sent to self. Duplicate turn execution is structurally impossible: only one actor instance exists at any point.

Information hiding (ring bearer location):

In Akka, the `RingBearerActor` would also be a `ClusterSingleton` owning the `trueRegion` field as private actor state. It would **never** publish `trueRegion` to any shared topic or shared actor mailbox. The message protocol would be defined such that `RingBearerActor` responds to light-side queries with the full position and to all other actors with only the public-facing `detectedRegion`. Since Akka actors communicate solely through typed message protocols (Akka Typed), and `trueRegion` is not a field in any message type sent to dark-side actors, the information boundary is **structurally enforced by the type system** — a dark-side component cannot receive `trueRegion` because there is no message type carrying it. In contrast to Go+Kafka where a routing bug can silently leak information, Akka Typed would produce a compile error if code attempted to construct a dark-side-bound message containing `trueRegion`.

5. Reflection

What Was the Hardest Part to Design

The **information boundary** between light-side and dark-side clients was consistently the hardest architectural concern throughout the project. The ring bearer exists simultaneously in two representations: an authoritative internal form carrying `trueRegion` (used by `TurnProcessor` to resolve interceptions) and a public stripped form carrying only `detectedRegion` (safe to broadcast to all clients). Maintaining this duality required explicit decisions at every layer.

The `EventRouter` goroutine must route `game.ring.position` to `lightSideSSECh` only, never to `darkSideSSECh`. The `game.broadcast` payload must be passed through `stripRingBearer()` before entering `darkSideSSECh`. `CacheManager` must maintain two separate cache views. Every HTTP API endpoint that returns unit or region data must call a cache helper that checks the requesting player's faction before deciding which representation to return. Forgetting any one of these check points would silently leak information — there is no compiler-level enforcement.

In contrast to an actor-model approach (see §4, Q3), where message type boundaries make it structurally impossible to send `trueRegion` to a dark-side actor, the Go+Kafka implementation relies entirely on programmer discipline and integration test coverage. This drove home the insight that **information hiding in distributed systems is a protocol-design problem, not just a code-organization problem**.

Distributed Ownership Complexity

The distributed turn execution design was the second-hardest concern. Three stateless Go instances behind a load balancer create an inherent split-brain risk: if all three observe the `turnTicker`, all three might attempt to advance the turn. The leader heartbeat mechanism solves this correctly — only the leader's `turnTicker` case executes the turn — but the design surface is large.

The leader heartbeat produces to `game.session` (compacted), which all instances consume. The heartbeat period (3 s) and the election timeout (two missed heartbeats → ~6–7 s) were chosen to minimize the window during which no leader is active, while not being so aggressive that normal GC pauses trigger spurious elections. Getting these timing constants right required careful testing under simulated partition scenarios.

The `GameOver` flag stored in the compacted `game.session` topic was a late addition: without it, a leader crash immediately after the winning turn produce (but before the heartbeat confirm) would cause the new leader to re-run the winning turn and produce a second `GameOver` event. The compacted flag ensures idempotence regardless of leader handover timing.

What Worked Well

State recovery via Kafka replay worked extremely well. After a clean shutdown and restart, any Go instance replays `game.events.*` from the beginning (or from a snapshot offset if available) and reconstructs `WorldStateCache` exactly. This eliminated the need for a separate database layer entirely — Kafka is the database. The design also made integration testing straightforward: a test could produce a known sequence of events to a test-mode Kafka cluster and verify that the resulting `WorldStateCache` matched expectations deterministically.

The pipeline pattern (§2.10, §2.11) for analysis tasks gave clean separation between the CPU-bound analysis work and the event-driven main loop. The `sync.WaitGroup` + `context.Context` timeout

pattern ensured that analysis tasks never blocked the main select loop even if they ran long.

What Would Be Redesigned

If restarting, we would **separate the write-side turn engine from the HTTP read-side more cleanly**. Currently, both share the same Go binary, which means HTTP handler latency can compete with turn-processing latency during busy turns. A cleaner split would run TurnProcessor as a dedicated process (or Kubernetes pod) consuming only from `game.orders.validated` and producing to `game.events.*`, with the HTTP/SSE process acting as a pure read-side projection service. This would align better with the CQRS pattern and allow independent scaling of the read and write sides.

We would also **add more integration tests around Kafka replay**, specifically around the leader handover scenario. The current test suite validates the happy path (single instance, replay from offset 0) but does not systematically test the case where a new leader starts mid-turn with an uncommitted work consumer offset. Stronger automated evidence for failover correctness would be the most impactful next improvement given the project's current state.

Overall, the current implementation is **playable and testable**: a real game can be run to completion, the information boundary holds under manual testing, and the leader election mechanism survives forced instance kills. The gap between the current state and production-readiness lies primarily in automated failover tests and cleaner write/read separation.

6. LLM Usage Log

#	Date	Tool / Model	Prompt Summary	Output Used	Modified?
1	2026-05-10	ChatGPT-5	"Design a Kafka topic schema for a turn-based ring-bearer game with information hiding between two player factions"	Initial topic list (8 topics); partition key suggestions	Yes — added <code>game.ring.detection</code> (2 partitions) and <code>game.dlq</code> ; revised retention values
2	2026-05-12	ChatGPT-5	"How should I implement leader election in Go using a Kafka compacted topic as a coordination primitive?"	Heartbeat loop pattern using <code>game.session</code> produce + consume; suggested manual offset commit strategy	Yes — adapted timeout constants; added sticky <code>GameOverFlag</code> logic not present in LLM output
3	2026-05-14	GitHub Copilot	Inline completion while writing EventRouter routing table	Suggested switch event.Topic skeleton with <code>lightSideSSECh</code> and <code>darkSideSSECh</code> cases	Yes — LLM did not include <code>stripRingBearer()</code> call; added information-boundary enforcement manually

#	Date	Tool / Model	Prompt Summary	Output Used	Modified?
4	2026-05-16	ChatGPT-5	"Write a Kafka Streams topology in Java that validates game orders against KTable snapshots and routes failures to a DLQ"	Java Kafka Streams skeleton for Topology 1 with 6 of 8 rules	Yes — translated to project context; added <code>DUPLICATE_UNIT_ORDER</code> and <code>ABILITY_ON_COOLDOWN</code> rules; adjusted KTable join semantics
5	2026-05-18	GitHub Copilot	Inline completion while writing Pipeline 1 dispatcher	Suggested <code>workCh + sync.WaitGroup + context.WithTimeout</code> pattern	Minimal modification — adjusted buffer size from 10 to 20 based on load testing
6	2026-05-20	Claude Sonnet 4.6	"Compare Go+Kafka vs Akka for distributed turn-based game architecture — focus on singleton ownership and information hiding"	Comparison text covering <code>ClusterSingleton</code> and <code>Akka Typed</code> message boundaries	Yes — used as basis for §4 Q3; rewritten substantially to be project-specific and accurate
7	2026-05-22	Claude Sonnet 4.6	"Write a reflection section for a distributed systems report covering what was hardest in Go+Kafka implementation"	Draft reflection (~150 words)	Yes — expanded to >300 words; added <code>GameOver</code> flag detail, pipeline pattern assessment, and redesign recommendations not present in draft